```
# Task 1

number = 5

if number % 2 == 0:
  print("Number is Even")
else:
  print("Number is Odd")
```

```
Number is Odd
```

```
# Task 2

x = ['alphabets', 'numerics', 'data types', '123', 'wild' ,'abc', 'chars']

for a in x:
  length = len(a)

  if length % 2 == 0:
    print(f"{a} is even")
  else:
    print(f"{a} odd")
```

```
alphabets odd
numerics is even
data types is even
123 odd
wild is even
abc odd
chars odd
```

```
# Task 3

greeting = 'hello'

print(greeting[::-1])
```

```
olleh
```

```
# Task 4

def func():
    result = []
    for a in range(0, 100):
      if a % 2 == 0 and a % 5 != 0:
        result.append(a)
    return result

l1= func()
print('List of multiples of 2 but not 5:',)
print(l1)
```

```
List of multiples of 2 but not 5:
[2, 4, 6, 8, 12, 14, 16, 18, 22, 24, 26, 28, 32, 34, 36, 38, 42, 44, 46, 48, 52, 54, 56, 58, 62, 64, 66, 68, 72, 74, 76, 78, 82, 84, 86, 88, 92, 94, 96, 98]
```

```
# Task 5

str = "Hello Mr. Rogers, how are you this fine Tuesday?"

def enum_cases(string):
  upper_case = 0
  lower_case = 0

  for a in string:
    if a.isupper():
      upper_case += 1
    elif a.islower():
      lower_case += 1
  return upper_case, lower_case

print(enum_cases(str))
```

```
(4, 33)
```

```
# Task 6

def func():
  for a in range (1, 101):
    if a % 3 == 0 and a % 5 == 0:
      print("FizzBuzz")
```

```
    elif a % 3 == 0:
      print("Fizz")
    elif a % 5 == 0:
      print("Buzz")
    else:
      print(a)

func()
```

```
1
2
Fizz
4
Buzz
Fizz
7
8
Fizz
Buzz
11
Fizz
13
14
FizzBuzz
16
17
Fizz
19
Buzz
Fizz
22
23
Fizz
Buzz
26
Fizz
28
29
FizzBuzz
31
32
Fizz
34
Buzz
```

```
Fizz
37
38
Fizz
Buzz
41
Fizz
43
44
FizzBuzz
46
47
Fizz
49
Buzz
Fizz
52
53
Fizz
Buzz
56
Fizz
```

```
# Task 7

l = [1, 21, 3, 14, 5, 60, 7, 6]
k = 81

def calculatesum(l, k):

  for i in range(len(l)):
    for j in range(i+1, len(l)):
      if l[i] + l[j] == k:
        return l[i], l[j]

  return None

print(calculatesum(l, k))
```

```
(21, 60)
```

```
# Task 8

students = {
```

```
      "Ali": {"age": 6, "address": "Lahore"},
      "Ahmad": {"age": 15, "address": "Islamabad"},
      "Eesha": {"age": 21, "address": "Karachi"},
}


def find_students(address, students):
  names = []

  for a, b in students.items():
    if b["address"] == address:
      names.append(a)

  names.sort()
  return names

find_students("Lahore", students)
```

```
['Ali']
```

```
# Task 9

def first_missing_positive(nums):
  num = 1
  while num in nums:
    num += 1

  return num

a = [7, 8, 9, 11]
first_missing_positive(a)
```

```
1
```

```
# Task 10

nested_dict = {
    'a': 1,
    'b': {
        'c': 2,
        'd': {
            'e': 3,
```

```
            'f': 4
        }
    },
    'g': 5
}

def flatten_dict(d, parent_key = '', sep = '_'):
  items = {}

  for k, v in d.items():
    new_key = f"{parent_key}{sep}{k}" if parent_key else k

    if type(v) == dict:
      items.update(flatten_dict(v, parent_key=new_key, sep='_'))
    else:
      items[new_key] = v

  return items

print(flatten_dict(nested_dict))
```

```
{'a': 1, 'b_c': 2, 'b_d_e': 3, 'b_d_f': 4, 'g': 5}
```

```
# Task 11

scores = [("Ali", 85), ("Ahmad", 92), ("Aisha", 78)]

def top_student(students):
  name = students[0][0]
  max_score = students[0][1]

  for a, b in students:
    if b > max_score:
      max_score = b
      name = a

  return name

top_student(scores)
```

```
'Ahmad'
```

```
# Task 12

text = "Hello, how are you today?"

def count_vowels(s):

  vowels_count = {'a':0,
                  'e': 0,
                  'i':0,
                  'o':0,
                  'u':0}

  s = s.lower()

  for char in s:
    if char in vowels_count:
      vowels_count[char] += 1

  return vowels_count


count_vowels(text)
```

```
{'a': 2, 'e': 2, 'i': 0, 'o': 4, 'u': 1}
```

```
# Task 13

numbers = [12, 3, 45, 7, 1, 67, 9]

def remove_outliers(lst):
  max_number = max(lst)
  min_number = min(lst)

  lst.remove(min_number)
  lst.remove(max_number)

  return lst

remove_outliers(numbers)
```

```
[12, 3, 45, 7, 9]
```

```
# Task 14
```

```
# Task 14

nums = [100, 4, 200, 1, 3, 2, 101, 99, 102, 103, 104]

num1 = sorted(list(set(nums)))

# print(num1)

def longest_consecutive_sequence(num2):

  max_seq = 1
  current_seq = 1

  for x in range(1, len(num2)):
    if num2[x] == num2[x-1]+1:
      current_seq += 1
    else:
      current_seq = 1

    if current_seq > max_seq:
      max_seq = current_seq
  return max_seq

longest_consecutive_sequence(num1)
```

```
6
```

```
# Task 15

class Trie:
    def __init__(self):
        # Empty dictionary se start karein
        self.root = {}

    def insert(self, word):
        node = self.root
        for char in word:
            # Agar letter pehle se nahi hai, to nayi dict bana dein
            if char not in node:
                node[char] = {}
            node = node[char]
        # Word ke aakhir me `#` lga kar mark kar dein ke word complete ho gaya
        node['#'] = True
```

```
    def search(self, word):
        node = self.root
        for char in word:
            if char not in node:
                return False
            node = node[char]
        # Check karein kya word complete wala `#` marker majood hai
        return '#' in node

    def startsWith(self, prefix):
        node = self.root
        for char in prefix:
            if char not in node:
                return False
            node = node[char]
        # Agar saare letters mil gaye to True
        return True


trie = Trie()
trie.insert("apple")
print(trie.search("apple"))   # True
print(trie.search("app"))     # False
print(trie.startsWith("app")) # True
trie.insert("app")
print(trie.search("app"))     # True
```

```
True
False
True
True
```

```
# Task 16

class CircularBuffer:
    def __init__(self, size):
        self.size = size
        self.buffer = []

    def append(self, x):
        # Agar buffer full hai, to sab se purana (first) element nikal dein
        if len(self.buffer) == self.size:
            self.buffer.pop(0)
```

```
        # Naya element add kar dein
        self.buffer.append(x)

    def get(self):
        # Buffer ko list ke roop me return kar dein
        return self.buffer


cb = CircularBuffer(3)
cb.append(1)
cb.append(2)
cb.append(3)
print(cb.get())  # [1, 2, 3]
cb.append(4)
print(cb.get())  # [2, 3, 4]
```

```
[1, 2, 3]
[2, 3, 4]
```

```
[[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]
 [1. 0. 0.]]
```

```
# Task 17
import numpy as np
def one_hot_encode(labels):
    # 1. Unique labels ki list banayein (order maintain rakhte hue)
    unique_labels = list(dict.fromkeys(labels))

    encoded = []
    # 2. Har label ke liye one-hot vector banayein
    for label in labels:
        row = [1.0 if label == category else 0.0 for category in unique_labels]
        encoded.append(row)

    # 3. List ko NumPy array me convert kar dein
    return np.array(encoded)

# Test Code
labels = ["cat", "dog", "bird", "cat"]
print(one_hot_encode(labels))
```